Acceso a datos en aplicaciones reactivas

Tecnologías clásicas

- Java proporciona dos tecnologías para acceder a bases de datos relacionales: JDBC y JPA

- Spring dispone de librerías para simplificar el uso de estas tecnologías: JdbcTemplate y Spring Data JPA

- Ambas son BLOQUEANTES

Antonio Martín Sierra

Reactividad en la capa DAO

- Para que una aplicación sea realmente reactiva, debe serlo en todas sus capas

- Diferentes librerías para conseguir reactividad en la capa de datos:

  - R2dbc. Permite generar flujos de datos a partir de una base de datos relacional, por ejemplo MySQL. Aun en estado inmaduro

  - Reactive Mongo Repository. Genera flujos de datos a partir de bases de datos Mongo DB. Integrado totalmente en Spring Boot

Antonio Maria Sierra

R2DBC

R2DBC. Dependencias

- **Librería para aplicar reactividad en una capa de datos que actúa sobre una base de datos relacional.**

- **Para utilizarla en aplicación Spring Boot, se deberá incluir el starter r2dbc y una implementación para el tipo de base de datos a utilizar:**

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-r2dbc</artifactId>
</dependency>
<dependency>
    <groupId>dev.miku</groupId>
    <artifactId>r2dbc-mysql</artifactId>
    <version>0.8.2.RELEASE</version>
</dependency>
```

Antonio Martín Sierra

El modelo de datos

- **La entidad se configura mediante anotaciones similares a las de JPA:**

```
import org.springframework.data.annotation.Id;
import org.springframework.data.relational.core.mapping.Column;
import org.springframework.data.relational.core.mapping.Table;

@Table("productos")
public class Producto {
    @Id
    @Column(value = "idProducto")
    private int idProducto;
    private String nombre;
    private String seccion;
    private double precio;
    :
```

Interfaz ReactiveCrudRepository

- **Similar a JpaRepository, si bien en este caso los métodos devuelven flujos (Flux y Mono).**

https://docs.spring.io/spring-data/commons/docs/current/api/org/springframework/data/repository/reactive/ReactiveCrudRepository.html

Modifier and Type	Method	Description
reactor.core.publisher.Mono<Long >	count()	Returns the number of entities available.
reactor.core.publisher.Mono<Void >	delete(T entity)	Deletes a given entity.
reactor.core.publisher.Mono<Void >	deleteAll()	Deletes all entities managed by the repository.
reactor.core.publisher.Mono<Void >	deleteAll(Iterable <? extends T> entities)	Deletes the given entities.
reactor.core.publisher.Mono<Void >	deleteAll(org.reactivestreams.Publisher<? extends T> entityStream)	Deletes the given entities supplied by a Publisher.
reactor.core.publisher.Mono<Void >	deleteAllById(Iterable <? extends ID> ids)	Deletes all instances of the type T with the given IDs.
reactor.core.publisher.Mono<Void >	deleteById(ID id)	Deletes the entity with the given id.
reactor.core.publisher.Mono<Void >	deleteById(org.reactivestreams.Publisher<ID> id)	Deletes the entity with the given id supplied by a Publisher.
reactor.core.publisher.Mono<Boolean >	existsById(ID id)	Returns whether an entity with the given id exists.
reactor.core.publisher.Mono<Boolean >	existsById(org.reactivestreams.Publisher<ID> id)	Returns whether an entity with the given id, supplied by a Publisher, exists.
reactor.core.publisher.Flux<T>	findAll()	Returns all instances of the type.
reactor.core.publisher.Flux<T>	findAllById(Iterable <ID> ids)	Returns all instances of the type T with the given IDs.
reactor.core.publisher.Flux<T>	findAllById(org.reactivestreams.Publisher<ID> idStream)	Returns all instances of the type T with the given IDs supplied by a Publisher.
reactor.core.publisher.Mono<T>	findById(ID id)	Retrieves an entity by its id.
reactor.core.publisher.Mono<T>	findById(org.reactivestreams.Publisher<ID> id)	Retrieves an entity by its id supplied by a Publisher.
<S extends T> reactor.core.publisher.Mono<S>	save(S entity)	Saves a given entity.

Métodos específicos

- **Spring implementa automáticamente métodos, a partir del nombre del mismo o instrucción SQL:**

Implementación a partir del nombre de método

Implementación a partir de la query

```
public interface AgendaDao extends ReactiveCrudRepository<Contacto, Integer>{

    Mono<Contacto> findByEmail(String email);

    @Query("Select * from Contacto  where edad<=?1")
    Flux<Contacto> buscarPorEdadMaxima(int edad);

    @Transactional
    @Query("Delete from Contacto Where email=?1")
    Mono<Void> eliminarPorEmail(String email);
}
```

Antonio

Configuración R2DBC

- **Los datos de conexión con la base de datos se configuran a través de una serie de propiedades en el archivo application.properties:**

```
spring.r2dbc.url=r2dbc:mysql://localhost:3306
spring.r2dbc.name=agenda
spring.r2dbc.username=root
spring.r2dbc.password=root
```

- **En la clase de configuración:**

paquete con la interfaz de repositorio

```
@EnableR2dbcRepositories(basePackages = {"dao"})
@SpringBootApplication
public class Application {
```

Antonio Mar

Reactive Mongo Repository

Reactive Mongo Repository

- **Librería para aplicar reactividad en una capa de datos que actúa sobre una base de datos Mongo DB.**

- **Para utilizarla en aplicación Spring Boot, se deberá incluir simplemente el siguiente starter:**

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-mongodb-reactive</artifactId>
</dependency>
```

Antonio Sierra

El modelo de datos

- **La entidad se configura mediante anotaciones para Mongo DB:**

```
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
@Document(collection = "cursos")
public class Curso {
    @Id
    private String id;
    private String nombre;
    private int duracion;
    private double precio;
    :
```

Interfaz ReactiveMongoRepository

- **Similar a ReactiveCrudRepository.**
- **Para definir una subinterfaz, se sigue el mismo proceso que en ReactiveCrudRepository, utilizando sintaxis de filtros Mongo DB:**

```
public interface CursosRepository extends ReactiveMongoRepository<Curso, Integer> {
    @Query("{'duracion':{$lt:?0}}")
    Flux<Curso> findByDuracionMax(int duracion);
    Mono<Curso> findByNombre(String nombre);
}
```

Configuración

- **La cadena de conexión a la base de datos se indica en application.properties:**

```
spring.data.mongodb.uri=mongodb://localhost:27017/formacion
```

- **En la clase de configuración:**

paquete con la interfaz de repositorio

```
@EnableReactiveMongoRepositories(basePackages = "com.curso.dao")
@SpringBootApplication
public class Application {
 :
}
```

Antonio Martín Sierra